

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_4

4. Abstract Factory Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Abstract Factory pattern.

GoF Definition

It provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Note The Abstract Factory pattern will make more sense to you if you understand both the Simple Factory pattern (Chapter [2](#)) and the Factory Method pattern (Chapter [3](#)). The Simple Factory pattern does not fall directly into the Gang of Four design patterns, so the discussion of that pattern appears in Part I of the book. I suggest you read those two chapters before you jump into this one.

Concept

An abstract factory is often referred to as a *factory of factories*. In this pattern, you provide a way to encapsulate a group of individual factories that have a common theme. In this process, you do not instantiate a class directly; instead, you instantiate a concrete factory and thereafter create products using the factory.

Note I suggest you read Q&A 4.3 to understand the differences between the Simple Factory, Factory Method, and Abstract Factory patterns.

You may see variations of client code when you see an implementation of this pattern. For example, you may see an optional factory provider that supplies the required factory to proceed further. Sometimes this provider is composed inside client code. I'll show you some of these variations.

In the upcoming example, you see that, inside the client code, you create a factory instance, and using this instance you create an animal:

```
// Making a wild dog and wild tiger through WildAnimalFactory
AnimalFactory animalFactory;
animalFactory = new WildAnimalFactory();
Dog dog = animalFactory.createDog("white");
Tiger tiger = animalFactory.createTiger("golden and cinnamon");
dog.displayMe();
tiger.aboutMe();
tiger.inviteDog(dog);
```

The previous code segment produces the following output:

```
You opt for a wild animal factory.
A wild dog with white color is created.
A wild tiger with golden and cinnamon color is created.
The wild dog says: I prefer to roam freely in jungles.Bow-Wow.
The wild tiger says: I prefer hunting in jungles.Halum.
The wild tiger says: I saw a wild dog in the jungle.
```

This pattern suits best when products are similar but the product families are different (for example, a domestic dog is quite different from a wild dog). This pattern helps you interchange specific implementations without changing the code that uses them, even at runtime. However, it may result in unnecessary complexity and extra work. Even debugging can become tough in some cases.

Real-Life Example

So what do I mean by the words “product families are similar (or different)?” Take the example of pet dogs and wild dogs. Similarly, you can see tigers in a zoo (or circus) and a jungle. Let’s call them pet tigers and wild tigers, respectively. Do you see anything in common? The first obvious thing is both pet dogs and wild dogs are dogs. Similarly, both pet tigers and wild tigers are tigers. But notice that there is another thing in common among them: both wild dogs and wild tigers love to roam in a jungle. Similarly, both pet tigers and pet dogs love to stay in a secure place where the food supply is guaranteed. So, we can say that the pet animals form one set and the wild animals form another set. The abstract factory pat-

tern suits best when you deal with a similar scenario. In the upcoming demonstration, you will see such an implementation.

Consider another example. Suppose you are decorating your room with two different types of tables; one is made of wood and the other one of steel. For the wooden type, you need to visit a carpenter, and for the steel type, you may need to go to a metal shop. Both are table factories. So, based on demand, you decide what kind of factory you need.

Computer World Example

Here are some examples:

- The `newInstance()` method of `javax.xml.parsers.DocumentBuilderFactory` is an example of an Abstract Factory pattern in JDK. The `newInstance()` method is static, but it returns the factory class considering other things, such as classpath scanning, external property, and so on. You can also change the factory class even though it is a static method. When you see the client code variation 2 in this chapter and analyze its uses, you'll find the similarity.
- If you are familiar with the C# programming language, you may notice that `ADO.NET` has already implemented similar concepts to establish a connection to a database.

Implementation

Wikipedia describes a typical structure of this pattern, which is similar to Figure [4-1](#) (https://en.wikipedia.org/wiki/Abstract_factory_pattern).

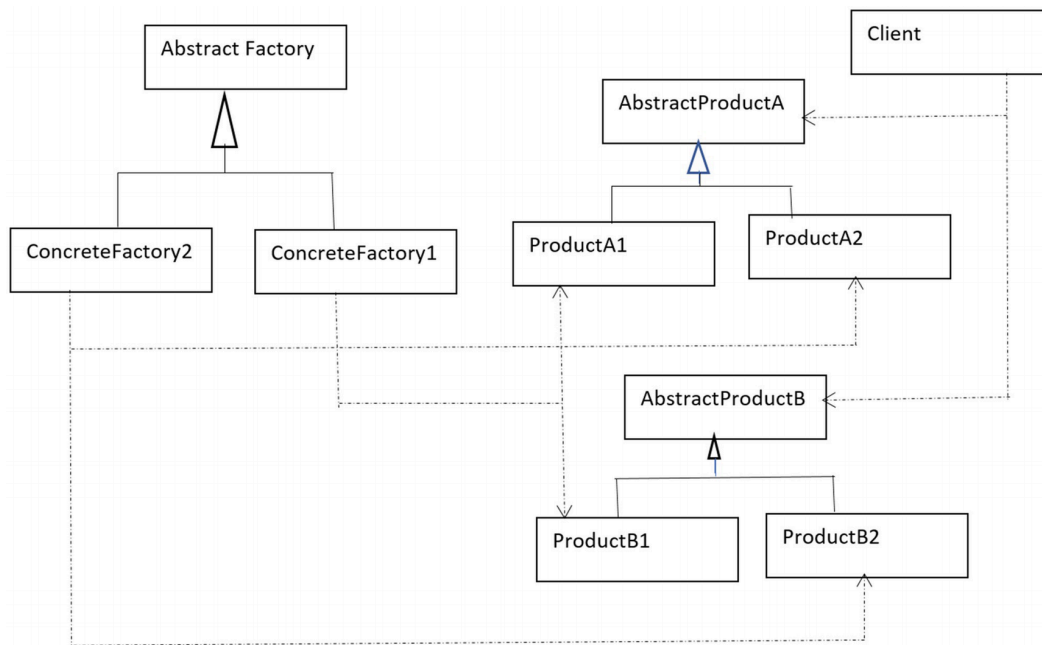


Figure 4-1 Abstract Factory pattern

You will follow a similar structure in the implementation in this chapter. In this example, there are two types of animals: pet animals and wild animals. `Client.java` is the client that is looking for some animals (they are wild dogs, pet dogs, wild tigers, and pet tigers). You'll explore the construction processes of both pet and wild animals in this implementation.

In this implementation, `AnimalFactory` is an abstract factory. Two concrete factories called `WildAnimalFactory` and `PetAnimalFactory` inherit from this abstract factory. These concrete factories are responsible for creating concrete products for dogs and tigers. As per their names, `WildAnimalFactory` creates wild animals (wild dogs and wild tigers) and `PetAnimalFactory` creates pet animals (pet dogs and pet tigers). For your reference, the participants and their roles are summarized here:

- **AnimalFactory:** Abstract factory. It is an abstract class in this implementation.
- **WildAnimalFactory:** It is a concrete factory and extends `AnimalFactory`. It creates wild dogs and wild tigers.
- **PetAnimalFactory:** Another concrete factory that also extends `AnimalFactory`, but this factory creates pet dogs and pet tigers.
- **Tiger and Dog:** Abstract products in this case. They are represented using two different Java interfaces.
- **PetTiger, PetDog, WildTiger, and WildDog:** They are the concrete products in this example. Both `PetTiger` and `WildTiger` implement the `Tiger` interface. Similarly, `PetDog` and `WildDog` implement the `Dog` interface. The `Dog` and `Tiger` interfaces have a different number of methods.

- I want you to note one interesting point. In many implementations, you may see that each concrete product has an equal number of methods. But you should not assume that this is mandatory. This is why, in the implementation, you see that tigers have an additional method called `inviteDog()`. This method represents a relationship between a tiger and a dog. To make this simple, you just use a simple message such as “The wild tiger says: I saw a wild dog in the jungle.”
- You’ll see variations of client code in this chapter. In variation 2, you’ll see the `FactoryProvider` inside the client code as follows:

```
animalFactory = FactoryProvider.getFactory("wild");  
Dog dog = animalFactory.createDog("white");  
Tiger tiger = animalFactory.createTiger("golden and cinnamon");  
dog.displayMe();  
tiger.aboutMe();  
tiger.inviteDog(dog);
```

- From the bold formatted line in the previous code segment, you can see that you’re NOT directly instantiating the factory instance; instead, you’re using the static method `getFactory(...)` of the `FactoryProvider` class to get the factory instance.
- Finally, there’s another variation of client code where you use object composition.

For now, let’s concentrate on the first implementation. It is because, apart from the client code, the remaining parts will be common for the rest of the chapter.

Class Diagram

Figure [4-2](#) shows the class diagram.

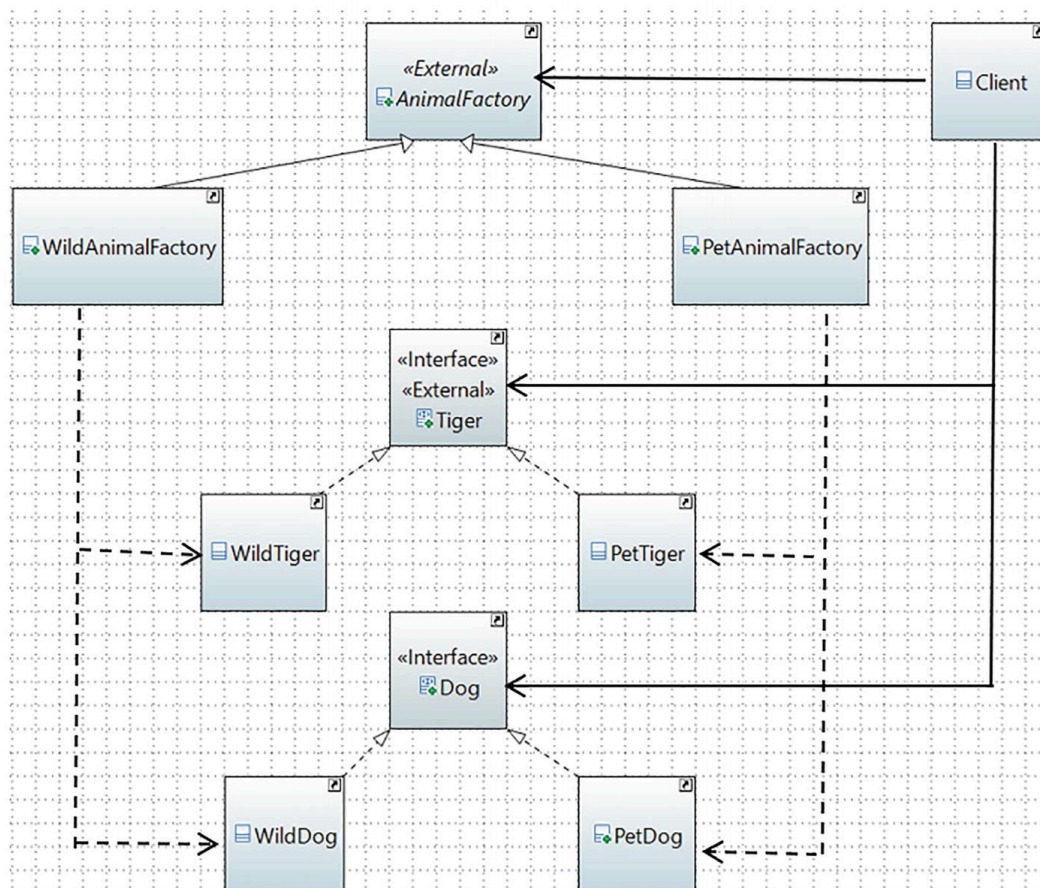


Figure 4-2 Class diagram

Package Explorer View

Figure 4-3 shows the high-level structure of the program. This program consists of many parts. I expand the important parts in this diagram.

abstractfactory

> pet

▼ wild

▼ WildAnimalFactory.java

▼ WildAnimalFactory

• WildAnimalFactory()

• createDog(String) : Dog

• createTiger(String) : Tiger

▼ WildDog.java

▼ WildDog

• WildDog(String)

• displayMe() : void

• toString() : String

▼ WildTiger.java

▼ WildTiger

• WildTiger(String)

• aboutMe() : void

• inviteDog(Dog) : void

• toString() : String

▼ AnimalFactory.java

> AnimalFactory

> Client.java

> Client2.java

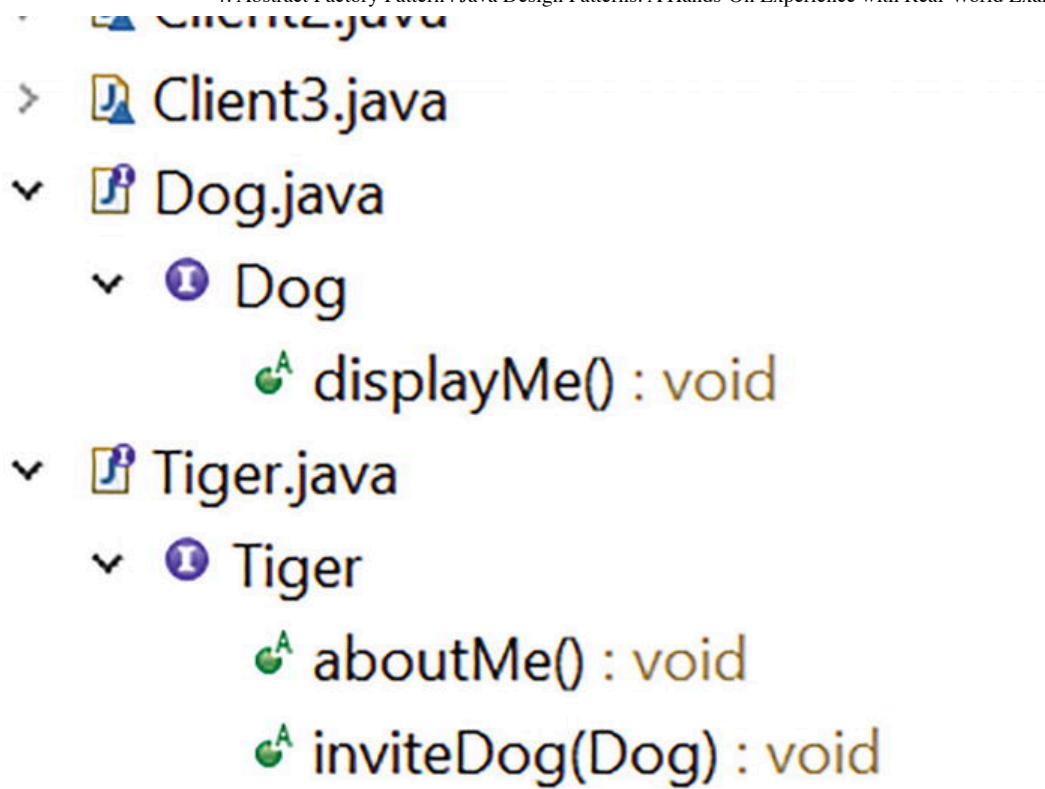


Figure 4-3 Package Explorer view

Demonstration 1

The complete code is inside the package: `jdp3e.abstractfactory`.

Here's the complete program:

```
// Abstract Factory.java
public abstract class AnimalFactory
{
    protected abstract Tiger createTiger(String color);
    protected abstract Dog createDog(String color);
}

// Abstract Product-1
// Tiger.java
public interface Tiger
{
    void aboutMe();
    void inviteDog(Dog dog);
}

// Abstract Product-2
// Dog.java
public interface Dog
{
    void displayMe();
}
```



```
// Concrete Factory 1: Wild animal factory
// WildAnimalFactory.java
import jdp3e.abstractfactory.AnimalFactory;
import jdp3e.abstractfactory.Dog;
import jdp3e.abstractfactory.Tiger;
public class WildAnimalFactory extends AnimalFactory {
    public WildAnimalFactory() {
        System.out.println("You opt for a wild animal factory.\n");
    }
    @Override
    public Tiger createTiger(String color) {
        return new WildTiger(color);
    }
    @Override
    public Dog createDog(String color) {
        return new WildDog(color);
    }
}

// Concrete product-A1
// WildTiger.java
import jdp3e.abstractfactory.Dog;
import jdp3e.abstractfactory.Tiger;
class WildTiger implements Tiger {
    public WildTiger(String color) {
        System.out.println("A wild tiger with " + color + "
                           color is created.");
    }
    @Override
    public void aboutMe() {
        System.out.println("The " + this + " says: I prefer
                           hunting in jungles.Halum.");
    }
    @Override
    public void inviteDog(Dog dog) {
        System.out.println("The " + this + " says: I saw a
                           " + dog + " in the jungle.");
    }
    @Override
    public String toString() {
        return "wild tiger";
    }
}

// Concrete product-B1
// WildDog.java
import jdp3e.abstractfactory.Dog;
class WildDog implements Dog {
```

```

        public WildDog(String color) {
            System.out.println("A wild dog with " + color + "
                               color is created.");
        }
        @Override
        public void displayMe() {
            System.out.println("The " + this + " says: I prefer
                               to roam freely in jungles. Bow-Wow.");
        }
        @Override
        public String toString() {
            return "wild dog";
        }
    }

    // Concrete Factory2: Pet animal factory
    // PetAnimalFactory.java
    import jdp3e.abstractfactory.AnimalFactory;
    import jdp3e.abstractfactory.Dog;
    import jdp3e.abstractfactory.Tiger;

    ;
    public class PetAnimalFactory extends AnimalFactory {
        public PetAnimalFactory() {
            System.out.println("You opt for a pet animal factory.\n");
        }
        @Override
        public Tiger createTiger(String color) {
            return new PetTiger(color);
        }
        @Override
        public Dog createDog(String color) {
            return new PetDog(color);
        }
    }

    // Concrete product-A2
    // PetTiger.java
    import jdp3e.abstractfactory.Dog;
    import jdp3e.abstractfactory.Tiger;
    class PetTiger implements Tiger {
        public PetTiger(String color) {
            System.out.println("A pet tiger with " + color + "
                               color is created.");
        }
        public void aboutMe() {
            System.out.println("The " + this + " says: Halum. I

```

```

        play in an animal circus.");
    }
    public void inviteDog(Dog dog) {
        System.out.println("The " + this + " says: I saw a
            " + dog + " in my town.");
    }
    @Override
    public String toString() {
        return "pet tiger";
    }
}
// Concrete product-B2
// PetDog.java
import jdp3e.abstractfactory.Dog;
class PetDog implements Dog {
    public PetDog(String color) {
        System.out.println("A pet dog with " + color + "
            color is created.");
    }
    @Override
    public void displayMe() {
        System.out.println("The " + this + " says:
            Bow-Wow. I prefer to stay at home.");
    }
    @Override
    public String toString() {
        return "pet dog";
    }
}
// Client.java
import jdp3e.abstractfactory.pet.PetAnimalFactory;
import jdp3e.abstractfactory.wild.WildAnimalFactory;
class Client {
    public static void main(String[] args) {
        System.out.println("***Abstract Factory Pattern Demo.***\n")
        AnimalFactory animalFactory;
        // Making a wild dog and wild tiger through

        // WildAnimalFactory
        animalFactory = new WildAnimalFactory();
        Dog dog = animalFactory.createDog("white");
        Tiger tiger = animalFactory.createTiger("golden and cinnamon
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
    }
}

```

```

        System.out.println("\n*****\n");
        // Making a pet dog and pet tiger through
        // PetAnimalFactory now.
        animalFactory = new PetAnimalFactory();
        dog = animalFactory.createDog("black");
        tiger = animalFactory.createTiger("yellow");
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
    }
}

```

Output

Here's the output:

```

***Abstract Factory Pattern Demo.***
You opt for a wild animal factory.
A wild dog with white color is created.
A wild tiger with golden and cinnamon color is created.
The wild dog says: I prefer to roam freely in jungles.Bow-Wow.
The wild tiger says: I prefer hunting in jungles.Halum.
The wild tiger says: I saw a wild dog in the jungle.
*****
You opt for a pet animal factory.
A pet dog with black color is created.
A pet tiger with yellow color is created.
The pet dog says: Bow-Wow. I prefer to stay at home.
The pet tiger says: Halum. I play in an animal circus.
The pet tiger says: I saw a pet dog in my town.

```

Analysis

You may notice that you cannot access the `PetDog` or `WildDog` classes inside the client code. This is because they are not public and imported.

The Client Code Variations

I told you that you will see variations in client code in different implementations of this pattern. In demonstration 1, you saw such a variation. Next I present two more.

Demonstration 2

Here you see the usage of an optional factory provider. It is used to avoid a dependency between the client code and the factories. This factory provider has a static method to provide the desired factory object to the client. Here is a sample implementation:

```
package jdp3e.abstractfactory;
import jdp3e.abstractfactory.pet.PetAnimalFactory;
import jdp3e.abstractfactory.wild.WildAnimalFactory;
// Factory provider
// (It is optional for you)
class FactoryProvider {
    public static AnimalFactory getFactory(String
                                         factoryType) {
        if (factoryType.contains("wild")) {
            return new WildAnimalFactory();
        } else if (factoryType.contains("pet")) {
            return new PetAnimalFactory();
        } else {
            throw new IllegalArgumentException("You need
                                             to pass either wild or pet as argument.");
        }
    }
}

class Client2 {
    public static void main(String[] args) {
        System.out.println("***Abstract Factory Pattern
                           Demo.Variation-2.***\n");
        AnimalFactory animalFactory;
        // Making a wild dog and wild tiger through
        // WildAnimalFactory
        animalFactory = FactoryProvider.getFactory("wild");
        Dog dog = animalFactory.createDog("white");
        Tiger tiger = animalFactory.createTiger("golden and cinnamon");
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
        System.out.println("\n*****\n");
        // Making a pet dog and pet tiger through
        // PetAnimalFactory
        animalFactory = FactoryProvider.getFactory("pet");
        dog = animalFactory.createDog("black");
        tiger = animalFactory.createTiger("yellow");
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
    }
}
```

```

    }
}

```

Demonstration 3

Sometimes you see the use of composition in the client code. The following code segment shows such an implementation:

```

package jdp3e.abstractfactory;
import jdp3e.abstractfactory.pet.PetAnimalFactory;
import jdp3e.abstractfactory.wild.WildAnimalFactory;
class Client3 {
    AnimalFactory factory;
    public Client3(AnimalFactory factory) {
        this.factory = factory;
    }
    public AnimalFactory getFactory() {
        return factory;
    }
    public static void main(String[] args) {
        System.out.println("***Abstract Factory Pattern
                               Demo.Variation-3.***\n");
        AnimalFactory animalFactory;
        // Making a wild dog and wild tiger through
        // WildAnimalFactory
        Client3 client = new Client3(new
                                   WildAnimalFactory());
        animalFactory = client.getFactory();
        Dog dog = animalFactory.createDog("white");
        Tiger tiger = animalFactory.createTiger("golden and cinnamon");
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
        System.out.println("\n*****\n");
        // Making a pet dog and pet tiger through
        // PetAnimalFactory now.
        client = new Client3(new PetAnimalFactory());
        animalFactory = client.getFactory();
        dog = animalFactory.createDog("black");
        tiger = animalFactory.createTiger("yellow");
        dog.displayMe();
        tiger.aboutMe();
        tiger.inviteDog(dog);
    }
}

```



```
}
```

Q&A Session

4.1 I see that both the Dog and Tiger interfaces contain different number of methods. Is that intentional?

Yes. I used a different number of methods with different names so that you do not assume that these interfaces should have a same number of methods with similar names. This is a change in this third edition of this book.

The `inviteDog()` method inside the Tiger hierarchy is optional. But I use it to show you the relationship between wild animals and pet animals. This is why, in the output, you see the following lines:

```
The wild tiger says: I saw a wild dog in the jungle
```

```
.
```

and

```
The pet tiger says: I saw a pet dog in my town.
```

This kind of relationship is important when you create a group of objects that work together to form a kit (or set). This is why this pattern is also known as “Kit.”

4.2 What are the challenges of using an abstract factory like this?

Any change in the abstract factory will force you to propagate the modification to the concrete factories. If you follow the design philosophy that says **program to an interface, not to an implementation**, you need to prepare for this. This is one of the key principles that developers should always keep in mind. In most scenarios, developers do not want to change their abstract factories.

In addition, the overall architecture is complex, and this is why debugging can become very challenging in some cases.

4.3 How can you distinguish a Simple Factory pattern from a Factory Method pattern or an Abstract Factory pattern?

I discuss the differences between a Simple Factory pattern and a Factory Method pattern in Chapter 3 (See Q&A 3.6).

Let's revisit how client code uses these factories as shown in the following diagrams.

Here's the code snippet from Simple Factory pattern:

```
AnimalFactory factory = new AnimalFactory();
Animal animal = factory.createAnimal("dog");
animal.displayBehavior();
animal = factory.createAnimal("tiger");
animal.displayBehavior();
```

Figure 4-4 shows the Simple Factory pattern.

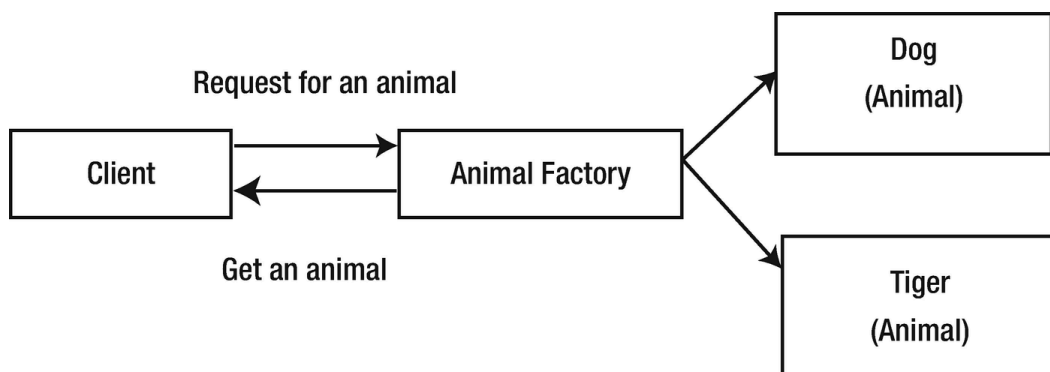


Figure 4-4 Simple Factory pattern

Here's the code snippet from the Factory Method pattern (taken from Demonstration 2):

```
AnimalFactory factory;
// Create a tiger and display its behavior
// using TigerFactory.
factory = new TigerFactory();
factory.createAndDisplayAnimal("yellow");
// Create a dog and display its behavior
// using DogFactory.
factory = new DogFactory();
factory.createAndDisplayAnimal("white");
```

Figure 4-5 shows the Factory Method pattern.

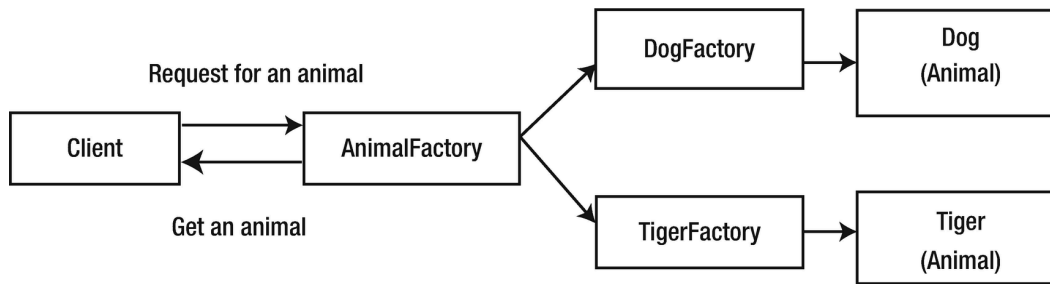


Figure 4-5 Factory Method pattern

Here's the code snippet from the Abstract Factory pattern (taken from client code variation 2):

```

AnimalFactory animalFactory;
AnimalFactory animalFactory;
// Making a wild dog and wild tiger through WildAnimalFactory
animalFactory = FactoryProvider.getFactory("wild");
Dog dog = animalFactory.createDog("white");
Tiger tiger = animalFactory.createTiger("golden and cinnamon");
dog.displayMe();
tiger.aboutMe();
tiger.inviteDog(dog);
System.out.println("\n*****\n");
// Making a wild dog and wild tiger through PetAnimalFactory
animalFactory = FactoryProvider.getFactory("pet");
dog = animalFactory.createDog("black");
tiger = animalFactory.createTiger("yellow");
dog.displayMe();
tiger.aboutMe();
tiger.inviteDog(dog);
  
```

Figure 4-6 shows the Abstract Factory pattern.

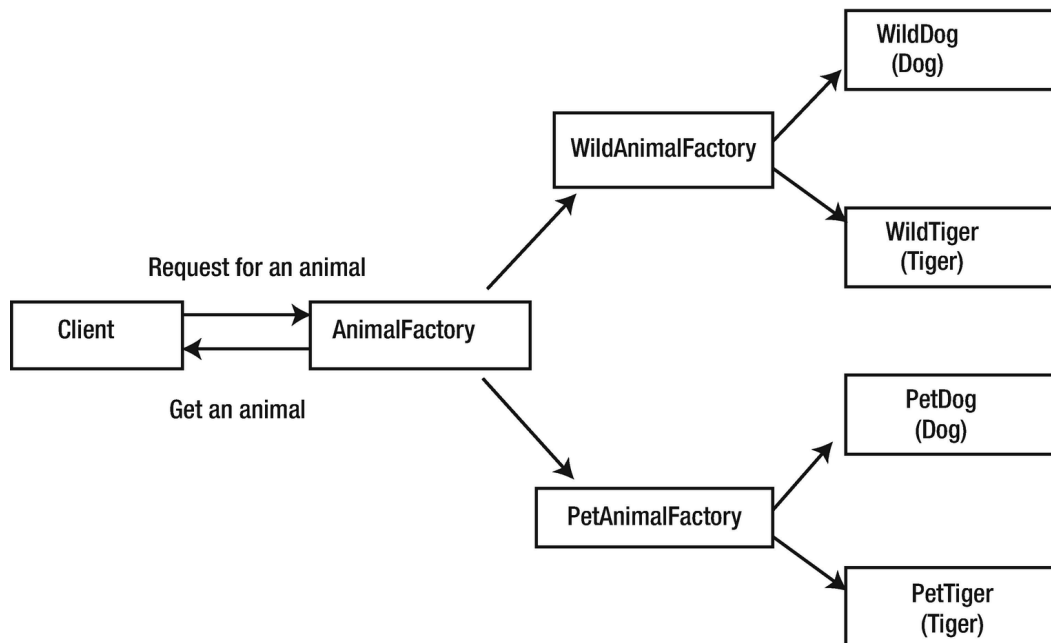


Figure 4-6 Abstract Factory pattern

In short, with the Simple Factory pattern, you can separate the code that will vary from the rest of the code (basically, you decouple the client code). This approach helps you to manage the code more easily. Another key advantage of this approach is that the client is unaware of how the objects are created. So, it promotes both security and abstraction.

However, this approach can violate the Open/Closed Principle (you saw this in Chapter 1). You can overcome this drawback using the Factory Method pattern, which allows subclasses to decide how the instantiation process will be completed. Put simply, you delegate the object creation to the subclasses that implement the factory method to create objects.

The abstract factory is basically a factory of factories. It creates a family of related objects, but it does not depend on the concrete classes. In this pattern, you encapsulate a group of individual factories that have a common theme. In this process, you do not instantiate a class directly; instead, you get a concrete factory (you use a provider for this in client code variation 2 and variation 3) and thereafter create products using the factory.

Lastly, I try to keep the examples simple. A factory method promotes inheritance, and its subclasses need to implement the factory method to create objects. The Abstract Factory pattern can promote object composition by creating the related objects using the methods that are exposed in a factory interface. In the end, all of these factories promote loose coupling by reducing the dependencies on concrete classes.

4.4 It would be helpful if you explain how the `newInstance()` method of `javax.xml.parsers.DocumentBuilderFactory` supports the concept of the Abstract Factory pattern.

Notice code segment 1. It is a code snippet to use the `newInstance()` method of `javax.xml.parsers.DocumentBuilderFactory`.

Code segment 1:

```
DocumentBuilderFactory factory;
// Get the factory
factory= DocumentBuilderFactory.newInstance();
// Get the builder
DocumentBuilder builder=factory.newDocumentBuilder();
// Making a use of the builder
Document doc=builder.newDocument();
// Some other code skipped
```

Now compare it with the following code (code segment 2). In the client code variation 2, you saw the following lines:

Code segment 2:

```
AnimalFactory animalFactory;
animalFactory = FactoryProvider.getFactory("wild");
Dog dog = animalFactory.createDog("white");
dog.displayMe();
```

If you carefully examine code segment 2, you will see the following steps:

- Get a factory.
- Make a product (`Dog`, in this case).
- Use the product.

These steps are similar to code segment 1. You may also notice the following:

- Both `AnimalFactory` and `DocumentBuilderFactory` are abstract classes.
- The `DocumentBuilder` is an abstract class and `Dog` is an interface [It means that you can use `Dog` as an abstract class too].
- The `getFactory()` and `newInstance()` methods are static.

I hope you see the similarities now.

Previous chapter

[< 3. Factory Method Pattern](#)

Next chapter

[5. Prototype Pattern >](#)